Team Reflection Journal: Preprocessing & Data Hygiene Workflow

Project Title: Clinical Stroke Risk Classification Pipeline

Repository: Stroke-Exploratory-Analysis

Dataset Source: Stroke Prediction Dataset (Kaggle)

Arinze Naphitaly Okafor (Team Lead)

Project Architecture & Leadership Overview

As the team lead, I architected our end-to-end data pipeline and established our version-controlled workspace to ensure full compliance with the assignment parameters. Working with the 5,110-observation Kaggle Stroke Prediction Dataset, my objective was building a reproducible, leakage-free Python preprocessing workflow in Google Colab while coordinating deliverables across our team repository.

What I Learned from Pipeline Architecture & Data Leakage Prevention

My biggest takeaway was understanding that data leakage is often subtle and occurs long before model fitting begins. If global statistics—such as dataset-wide means, scalers, or imputation parameters—are computed across the entire cohort prior to partitioning, holdout distribution knowledge inadvertently leaks into the feature space. To enforce strict data hygiene, I implemented a stratified 70/30 train-test split ($N = 3{,}576$ train, $N = 1{,}533$ test) as the very first operation. Preserving the exact ~4.9% positive stroke ratio across both partitions was critical. I ensured that all parameter calculations—specifically computing the training median BMI (27.90) and fitting StandardScaler on age, avg_glucose_level, and bmi—were learned strictly from the training matrix and statically applied to the holdout test fold without refitting.

Critical Evaluation of Class Balancing via SMOTE

Analyzing our baseline training fold revealed an extreme ~19:1 class disparity (3,402 non-stroke cases vs. 174 stroke cases). In clinical diagnostics, this distribution poses severe modeling

hazards: a naive classifier predicting zero for every patient achieves ~95.1% accuracy while failing 100% of high-risk patients. To correct this, I implemented Synthetic Minority Over-sampling Technique (SMOTE) exclusively on the training split, synthesizing minority instances to achieve a balanced 3,402 / 3,402 distribution. Critically, I left the holdout test partition untouched. Synthetically altering evaluation data would invalidate clinical reality; the model must ultimately be tested against authentic, unadjusted class prevalence.

Technical Takeaways & Next Steps

I learned that clean preprocessing requires systematic visual validation. By constructing before-and-after Matplotlib and Seaborn distribution suites, I confirmed that continuous variables were successfully standardized ($\mu \approx 0, \sigma \approx 1$) and categorical dimensions aligned across splits post-encoding. Our exported matrices (cleaned_train_stroke_data.csv and cleaned_test_stroke_data.csv) provide an uncompromised foundation. My immediate next step is leading the transition into supervised model benchmarking, configuring an imblearn pipeline to evaluate candidate algorithms against Recall and PR-AUC.

Hamza Ajmal

Project overview

I completed this project using the approved Stroke Prediction Dataset from Kaggle. The original dataset contains 5,110 rows and 12 columns. The goal was to prepare the data for a future supervised binary-classification exercise using Python in Jupyter Notebook.

What I learned from EDA

I learned to inspect the dataset structure, data types, missing values, and target distribution before deciding how to prepare the data. The training subset contained 3,577 rows. BMI had 148 missing values in the training data, and the stroke target was strongly imbalanced: 3,403 records had label 0 and 174 had label 1.

Train/test split and avoiding leakage

I used a reproducible 70/30 split in Python, resulting in 3,577 training rows and 1,533 testing rows. The preprocessing pipeline was fitted on training features and then applied to the holdout test features. This helps prevent information from the test set leaking into the preprocessing stage. The test set was not resampled.

Cleaning and feature preparation

The notebook imputes missing BMI values with the training median, applies a Yeo-Johnson power transform (normalization) and standard scaling to age, average glucose level and BMI, encodes ever_married and Residence_type as 0/1, one-hot encodes gender, work type and smoking status, and keeps 'Unknown' smoking status as its own category because about 30% of training rows have it. It excludes the ID column and creates an age–hypertension interaction feature. Normalization reduced skewness of glucose from 1.58 to 0.08 and BMI from 1.04 to 0.00. The processed feature table contains 20 columns, with no missing values after preprocessing.

Class imbalance

The original training labels were 3,403 records in class 0 and 174 in class 1. Random oversampling was applied only to the training data, producing 3,403 records in each class. This demonstrates class balancing while preserving the original, unbalanced holdout test data. I

learned that accuracy alone may be misleading for this dataset, so future models should also be evaluated with precision, recall, F1-score, a confusion matrix, and ROC-AUC.

Limitations and next steps

This notebook prepares data but does not establish that any model can reliably predict stroke in real-world settings. The dataset may not represent every population, and patterns in it do not prove causation. My next step would be to train and compare classification models using the prepared training data and evaluate them once on the untouched test set.

Jonathan Willturner

Project Quality Assurance & Evaluation Scope

My focus throughout this phase centered on post-transformation auditing, data integrity verification, and structuring our downstream evaluation framework. Working with the preprocessed stroke data, my objective was ensuring that transformations maintained statistical integrity without introducing mathematical anomalies or schema misalignments between partitions.

What I Learned from Data Verification & Integrity Auditing

Through comprehensive verification of the exported matrices, I learned how critical invariant boundary checks are in real-world machine learning pipelines. When categorical variables are converted via one-hot encoding across separate splits, differences in low-frequency category counts can cause silent column mismatches. I audited the column alignment across both splits to verify that edge categories (such as removing the single 'Other' entry in gender) produced an identical, consistent feature space without zero-variance columns. Furthermore, I verified null-

invariant properties across all columns, confirming that median imputation completely eliminated missingness without index shifts.

Critical Analysis of Evaluation Metrics & Modeling Pitfalls

A vital insight from this workflow is that standard evaluation metrics—specifically overall accuracy and standard ROC-AUC—are fundamentally misleading when applied to acute clinical events. In a population where stroke incidence is under 5%, an uncalibrated model that predicts negative for every instance attains ~95% accuracy while remaining entirely dysfunctional in a hospital triage setting. I learned that our evaluation criteria must prioritize Recall (Sensitivity) to penalize life-threatening False Negatives. Additionally, I identified that Precision-Recall AUC (PR-AUC) must supersede ROC-AUC, as PR-AUC evaluates performance strictly within the minority positive class rather than being inflated by massive True Negative counts.

Limitations & Future Modeling Strategy

While synthetic resampling resolves training bias, it can generate artificial feature combinations along boundary lines. My next step is designing a cross-validation framework utilizing imblearn.pipeline.Pipeline, ensuring that resampling occurs strictly within each cross-validation fold during hyperparameter tuning rather than globally across the training matrix. This ensures our upcoming model comparisons (Logistic Regression, Random Forests, and Gradient Boosting) yield statistically defensible diagnostic boundaries.